Lecture 4: Dictionary & Set in Python

NO MERCY Practice Questions

Instructions:
• Work through these questions in order
• Do not refer to solutions until you've attempted each question
• Focus on logic and problem-solving
• The last 3 questions are **NO MERCY** difficulty
• Submit your answers one at a time after attempting

Question 1: Dictionary Max Value

```
Given a dictionary where keys are product names and values are prices, find the
product with the highest price. If there's a tie (multiple products with the same
highest price), return the one that appears first in insertion order.

Example:
prices = {'apple': 1.5, 'banana': 0.8, 'orange': 1.5}
Output should be: 'apple'
```

Question 2: Word Frequency Counter

```
Write a function that takes a string and returns a dictionary where keys are
words and values are their frequencies. The string contains words separated by
spaces. Your function should handle edge cases like empty strings and should be
case-insensitive.

Example:
text = "Hello world hello Python"
Output should be: {'hello': 2, 'world': 1, 'python': 1}
```

Question 3: Set Intersection Edge Case

```
Given three sets, find elements that appear in at least two of the three sets. Be
careful with edge cases like empty sets or sets with only one element.

Example:
set1 = {1, 2, 3}
set2 = {2, 3, 4}
set3 = {3, 4, 5}
Output should be: {2, 3, 4}
```

Question 4: Merge Dictionaries with Conflict Resolution

Given two dictionaries, merge them such that if a key exists in both dictionaries, the value should be the sum of both values. Return the merged dictionary. Do not modify the original dictionaries.

Example:
dict1 = {'a': 5, 'b': 10}
dict2 = {'b': 3, 'c': 7}
Output should be: {'a': 5, 'b': 13, 'c': 7}

Question 5: Group by Value Type

Given a dictionary with mixed value types (integers, strings, floats), create a new dictionary where keys are the value types (as strings like 'int', 'str', 'float') and values are lists of keys whose values are of that type.

Example:
data = {'name': 'Alice', 'age': 30, 'height': 5.6, 'city': 'NYC', 'score': 95}
Output should be: {'str': ['name', 'city'], 'int': ['age', 'score'], 'float': ['height']}

Question 6: Symmetric Difference with Multiple Sets

Given a list of sets, find the "symmetric difference of all": elements that appear in an odd number of sets. (An element that appears in 1, 3, 5... sets should be included. An element that appears in 2, 4, 6... sets should not be included.)

Example:
sets_list = [{1, 2, 3}, {2, 3, 4}, {3, 4, 5}]
Output should be: {1, 5}

Question 7: Invert Dictionary with Multiple Keys

Given a dictionary, create an "inverted" dictionary where original values become keys and original keys become values in a list. Handle cases where multiple keys map to the same value.

Example:
```
data = {'a': 'x', 'b': 'y', 'c': 'x'}
Output should be: {'x': ['a', 'c'], 'y': ['b']}
```

Question 8: Find Common Subsequence Keys (NO MERCY)

Given two dictionaries (dict1 and dict2) with string keys, find a new dictionary containing only keys that are substrings of each other. If key 'abc' from dict1 is a substring of key 'xabcy' from dict2, include both keys with their respective values from their original dictionaries.

Example:
```
dict1 = {'cat': 1, 'dog': 2}
dict2 = {'concatenate': 10, 'doggy': 20, 'bird': 30}
Output should be: {'cat': (1, 10), 'dog': (2, 20)}
```

Hint: Consider case sensitivity and the direction of substring checking.

Question 9: Complex Set Chain (NO MERCY)

Given a dictionary where keys are set identifiers (strings) and values are sets of integers, determine which sets form a "chain": A sequence where set1 $\subset$ set2 $\subset$ set3... (proper subset relationship). Return the longest chain as a list of set identifiers in order.

Example:
```
sets = {'A': {1}, 'B': {1, 2}, 'C': {1, 2, 3}, 'D': {2, 3}}
Output should be: ['A', 'B', 'C']
```

Proper subset means: all elements of A are in B, but A $\neq$ B

Question 10: Build Dependency Graph (NO MERCY)

You have a dictionary where keys are task names and values are sets of task dependencies (tasks that must be completed first). Find the optimal order to complete all tasks such that dependencies are satisfied. If no valid order exists (circular dependency), return None. If multiple valid orders exist, return the one that is lexicographically smallest.

Example:
```
tasks = {'A': {'B', 'C'}, 'B': {'C'}, 'C': set(), 'D': {'A'}}
Output should be: ['C', 'B', 'A', 'D']
```

Explanation: C has no dependencies, so it's first. B depends on C, A depends on B and C, D depends on A. Lexicographically smallest valid order is shown above.

How to proceed:
Attempt each question and submit your solution when ready. I will strictly check your answer and provide feedback before moving to the next question.